

MLSys 2026 Scheduling Contest – Track A

Toni Boehnlein Pál András Papp

April 24, 2026

1 Overview

We treat the scheduling problem as a two-level search: an outer *partition* of operations into subgraphs, and an inner *coupling* layer that stitches adjacent subgraphs together via fast-memory retention. Both levels optimise the same roofline cost model, share a thread-safe memoisation cache, and are driven by a Fiduccia–Mattheyses (FM) local search embedded inside an evolutionary multi-start loop. Symmetry in the input DAG (parallel branches, repeated series motifs) is detected once via Merkle hashing and used to guide local search and crossover.

The binary targets the 2–120s timeouts in the contest harness: the time budget is parsed from the benchmark file name and strictly honoured by a single deadline propagated to every worker thread.

2 Cost model

The reference cost is re-implemented bottom-up in `src/core/subgraph.cpp`. For a candidate op-set we derive the tile-slice sizes by backward propagation from the boundary outputs, classify each boundary tensor as *once/row/col/tile/stream load* (based on which tile-index component drives it), and then sum a per-tile roofline $\max(\text{compute}, \text{memory})$. Split- k is handled exactly: the first and last k -steps pay the full per-tile I/O and the output evict, while the middle $(nk-2)$ steps only stream inputs. Snake traversal (RowMajor / ColMajor) is implemented as closed-form loop counts, not a per-tile simulation, so enumerating every feasible (w, h, k, snake) config is cheap.

Feasibility is a first-class concept. `Subgraph::create` refuses any op-set that has mixed internal/external consumers of the same tensor, inconsistent boundary-output dimensions, or a disconnected sub-DAG. The PW-sink rule ($k=1$ whenever a Pointwise produces a boundary output) is baked into the precomputed candidate set for k . Tiling divisibility ($w \mid W_{\text{out}}$, etc.) is precomputed as a fast-path intersection. These invariants are verified by 47 unit/integration tests covering each move and each cost-model branch.

3 Partition search

The partition layer minimises the sum of subgraph costs assuming no retention. The state is a set of groups (op subsets); moves are MERGE, STEAL, RECOMPUTE, EJECT, INTERNAL-EJECT, and SPLIT, plus two tensor-level operators restricted to the FM layer – TENSOR-MERGE (fuse all groups containing consumers of a shared input tensor) and TENSOR-EXTRACT (pull just the consumer ops into a new group). Each move validates acyclicity on the condensed group DAG before evaluation (modified Kahn with OR-semantics for recomputed ops) and ephemeral-gap preservation (mixed-consumer constraint).

3.1 FM with drift

An FM pass forms an active set from a small random seed of ops and then iterates: pop the best move across the active set, apply it, lock the moved op so it cannot initiate again this pass, and expand the active set with the neighbouring ops of the affected groups (which are re-evaluated with the locked set in mind). We accept worsening moves up to a *drift budget* $\varepsilon \cdot C_0$ below the best snapshot seen so far; once the cumulative gain drops beyond this bound the pass aborts. The outer loop restarts each subsequent pass from the running best partition, with a cosine-annealed drift fraction that decays from wide exploration at pass 0 to tight refinement near the end. Each pass also returns its perturbed end state so that the evolutionary layer can reseed from a still-drifting partition rather than always from a local minimum.

3.2 Evolutionary pool

Generation 0 seeds the pool from six structurally diverse initialisers (trivial, chain+edge, seed+grow, rev-topo, tensor-aligned, random) plus a symmetry-driven init that reuses detected parallel/series patterns. Each seed is refined by greedy + FM. Subsequent generations pick mutation (2/3) or crossover (1/3) tasks. Mutations are structural (random merges, splits, reassignments, block moves, tensor merges) and ignore cost, providing the diversity that the gain-guided FM cannot.

Crossover uses agreement clustering à la Sanders–Schulz: ops co-grouped in both parents form inherited clusters, the remainder is reassigned greedily. We additionally implement *DAG-cut* crossover which slices along topologically compatible boundaries and splices in a donor sub-partition, and a symmetry-aware variant that restricts donor candidates to Merkle-equivalent regions.

Pool admission is diversity-aware: partitions are compared via adjusted Rand index (ARI) on the contingency table, canonicalised by Merkle hashes so structurally symmetric partitions are collapsed. Insertion protects the best-cost entry, admits any partition that is either strictly better or sufficiently far from the nearest neighbour.

4 Symmetry detection

ML graphs are full of exact repetition (parallel heads, MoE experts, stacked transformer blocks) and reusing a good grouping across replicas is cheaper than re-searching for it. We precompute a directed Merkle fingerprint per op: a forward hash composed from the op’s init hash (type, base cost, I/O shapes) and the sorted forward hashes of its predecessors, iterated to a fixed point, plus a symmetric backward hash; the combined hash identifies ops whose full predecessor and successor cones are isomorphic. Equivalence classes of the forward hash are grown into parallel **SymmetricPatterns** via a two-phase orbit-level merger with a top/bottom level constraint to prevent cycles, and a separate scan of the topological order extracts **SeriesPatterns** – consecutive blocks of K ops with matching internal and inter-block wiring. These patterns feed four places in the search: **symm_init** replicates a solved partition from one component to all the others; a coverage-weighted symmetric-mutation operator edits every replica simultaneously so the child stays on the symmetric manifold; DAG-cut crossover restricts donor slices to Merkle-equivalent regions so the cut boundaries match; and **merkle_canonicalise** rank-matches ops within each class before ARI distance, so partitions that differ only by a symmetry permutation hash to distance 0.

5 Retention and coupling

Retention is modelled as a directed coupling edge $g_a \rightarrow g_b$ carrying a set of retained tensors. Each group has at most one incoming and one outgoing edge, so coupling chains are disjoint paths in the group DAG. The cost of a coupled pair recomputes both endpoints with retention context and caches the result in a second cost-cache tier (**evaluate_with_context**).

The coupled-partition layer runs its own FM/evo loop with all eight partition moves lifted to respect chains, plus five coupling-specific moves: **COUPLE/UNCUPLE** add or remove a coupling edge $g_a \rightarrow g_b$ carrying a tensor t ; **RETAIN-FORCE-SPLIT** splits a group at a bridge edge so that an internal tensor becomes a boundary output and immediately couples the two halves; **FORCE-RETAIN** splits the destination group so a coupling that would otherwise overflow fast memory becomes feasible; and **EPHEMERAL-FUSE** extracts a producer and one of its consumers into a fresh group where the shared tensor is ephemeral, and couples that group to the remaining consumer. Each move validates chain-level acyclicity via a lifted BFS reachability check before any cost evaluation.

Phase 2 seeds the coupling pool from ordering heuristics (DFS retain-score and beam-search ordering) applied to every partition in the Phase 1 pool, plus a random-coupling variant for diversity; Phase 3 lets the coupled FM evo explore from there until the deadline. The final solution falls back to the uncoupled best partition if the coupled candidate fails validation.

6 Engineering notes

Cost cache. Every partition move and every coupling move triggers a subgraph cost evaluation, so the cache is the hot path of the whole solver. We use a two-tier lock-free open-addressed table (**src/core/cost_cache.h**): the base tier keys $\text{hash}(\text{sorted op-set}) \rightarrow \text{CostResult}$ and serves all Phase 1 evaluations; the retention tier extends the key with (entering, retain) sets and serves the coupled search. Keys are stored as 64-bit wyhash-style fingerprints with no backing op-index vector – a correctness shortcut justified by a per-lookup collision probability below 10^{-9} at realistic populations – so a lookup is a single atomic load and writes are a CAS on a per-slot atomic hash. All threads and all phases share one cache instance; Phase 2/3 rely on Phase 1 having warmed the base tier. Hit rates are well above 90 % after the first generation, and the retention tier falls through to the base tier when entering and retain are both empty so coupled evaluations automatically reuse Phase 1 work.

Incremental indexing. The partition maintains an $\text{op} \rightarrow \text{group}$ index updated in-place on every move and a compact **GroupDAG** with $O(|E|)$ **merge_creates_cycle** checks. Dirty sets are propagated from the affected groups to their DAG neighbours so only impacted moves are re-evaluated, and the move-audit pattern (**doc/move_audit_pattern.md**) replays each move in a sandboxed copy before commit to catch any invariant violation at write time rather than discovering it downstream.

Parallelism. Init, FM, and evolution tasks run on up to 8 threads behind a work-stealing task queue; task granularity is a full FM outer loop, so lock contention on the diversity pool is negligible. All shared state (cost cache, pool) is either lock-free or coarse-grained mutex-guarded.

Safety margins. The budget is clamped to 95 % of the posted timeout and propagated as an absolute deadline, so every FM pass and cost evaluation checks wall time and exits cleanly before the harness kills the process.

7 Results

On the five released benchmarks (**mlsys-2026**–{1,5,9,13,17}) the solver returns valid solutions within the allotted time. The evolutionary loop keeps improving the best partition on the larger instances up to the 60s deadline; on small instances the pool converges quickly and the remaining budget is spent on coupling refinement.